Optimisation: The Engine of MPC

MSc Model Predictive Control — Chapter 2

İbrahim Küçükdemiral

Glasgow Caledonian University

March 24, 2026

A guiding thought

"Since the fabric of the universe is most perfect and the work of a most wise Creator, nothing at all takes place in the universe in which some rule of maximum or minimum does not appear."

— **Leonhard Euler**

Learning Objectives

After studying this chapter, you should be able to:

1. Formulate an engineering problem as a mathematical optimisation problem.
2. Identify the three components: decision variables, objective function, constraints.
3. Distinguish between convex and non-convex sets and functions.
4. Classify problems as LP, QP, or MILP and explain why the distinction matters.
5. Write and solve optimisation problems in MATLAB using YALMIP.
6. Explain why convexity guarantees that any local minimum is the global minimum.

Outline

1. Why optimisation is the engine of MPC
2. What is optimisation?
3. Illustrative examples
4. Multi-variable optimisation ingredients
5. Standard form and notation
6. Terminology: geometry and convexity
7. Classification of optimisation problems
   - Classification I: Existence of constraints
   - Classification II: Mathematical structure
8. Software tools for optimisation
9. From theory to code: an Economic MPC example
10. Key takeaways
11. Exercises for self-study

From reactive control to optimisation-based control

- A classical PID controller follows a simple, fixed rule: if the error increases, push back harder. It is reactive and rigid.
- An MPC controller does not follow a pre-written rule. At every single time step, it faces a choice and determines the best control action within the given constraints.
- This process of systematic selection is called *optimisation*.
- If the system model is the map that tells the controller where it *can* go, optimisation is the navigator that decides where it *should* go.

Core idea

Optimisation provides the mathematical framework for translating engineering objectives into computable control actions.

What does optimisation buy us in control?

- A clean way to encode trade-offs: tracking vs. effort vs. safety.
- A uniform way to handle constraints (actuators, states, safety envelopes).
- A natural language for multi-step decision making (horizon-based planning).
- A bridge between models (*where we can go*) and goals (*where we should go*).

Definition and standard form

Mathematical Definition of Optimisation

A numerical approach for seeking a **best** decision or action from a **set of alternatives**.

Generic optimisation problem:

$$\begin{aligned} \min_{x} \quad & J(x) \\ \text{s.t.} \quad & x \in S \end{aligned}$$

The anatomy of an optimisation problem

Almost every optimisation problem has three ingredients:

1. **Decision variables** (x): knobs the solver can turn.
2. **Objective function** (J): scalar performance score to minimise.
3. **Constraints**: rules defining feasibility (S).

In MPC

Decision variables are typically a *sequence* of future inputs over the prediction horizon.

Constraints: inequality vs equality

$$
\begin{aligned}
\min_{x} \quad & J(x) \\
\text{s.t.} \quad & g_i(x) \leq 0, \quad i = 1, \ldots, m \qquad \text{(inequalities)} \\
& h_j(x) = 0, \quad j = 1, \ldots, q \qquad \text{(equalities)}
\end{aligned}
$$

Study note: Constraint Types

- Inequalities describe ranges: "speed ≤ 100 km/h".
- Equalities describe strict physics: e.g. $F = ma$.
- In MPC, system dynamics ($x_{k+1} = Ax_k + Bu_k$) are treated as **equality constraints**.

Modelling: from story to maths

Turning a physical/data-driven problem into optimisation form is **modelling**.

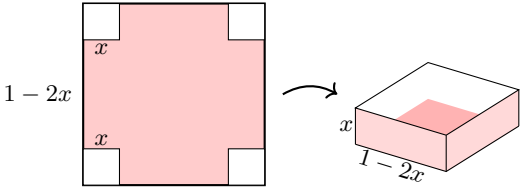
A useful systematic approach:

1. Identify decision variables (what can we choose?)
2. Formulate the objective (what means "best"?)
3. Define constraints (what must be obeyed?)

Example 1: Maximising box volume (setup)

Problem Statement

A $1\text{m} \times 1\text{m}$ square sheet. Cut squares of side x from corners and fold into an open-top box. Maximise volume.



Example 1: Maximising box volume (model)

Decision variable: x (corner cut size).
Constraints:

- $x \geq 0$ (cut size non-negative)
- $2x \leq 1 \Rightarrow x \leq 0.5$ (cuts cannot overlap)

Objective (volume):

$$V(x) = \underbrace{(1-2x)^2}_{\text{base area}} \cdot \underbrace{x}_{\text{height}} = x(1-2x)^2$$

Standard form trick

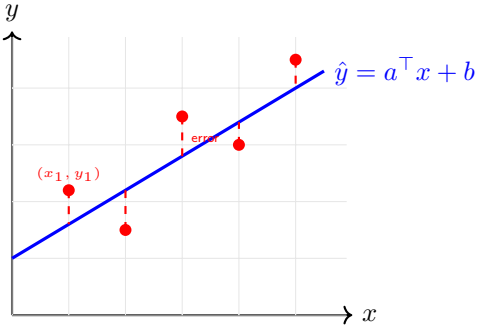
Maximising $f(x)$ is equivalent to minimising $-f(x)$ (so we can use minimisation solvers).

$$\begin{array}{ll} \max\limits_{x} & x(1-2x)^2 \\ \text{s.t.} & 0 \leq x \leq 0.5 \end{array} \quad \equiv \quad \begin{array}{ll} \min\limits_{x} & -x(1-2x)^2 \\ \text{s.t.} & 0 \leq x \leq 0.5 \end{array}$$

Example 2: Linear regression (least squares)

Problem Statement

Given data $\{(x_i, y_i)\}_{i=1}^{N}$, fit a line that "best" matches the data.



Example 2: Linear regression (model)

In this problem, the x_i are **data**, not decision variables.
Decision variables: slope vector $a \in \mathbb{R}^n$ and intercept $b \in \mathbb{R}$.
Prediction: $\hat{y}_i = a^\top x_i + b$.
Objective: minimise the sum of squared errors:

$$\min_{a,b} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2 = \min_{a,b} \sum_{i=1}^{N} \left(y_i - (a^\top x_i + b)\right)^2$$

Constraints: none (unconstrained).

The decision vector

In engineering, we usually choose many values at once.

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \in \mathbb{R}^n$$

- The objective maps vector $\to$ scalar: $J : \mathbb{R}^n \to \mathbb{R}$.
- In MPC: if horizon $N = 10$ and 2 inputs, decision dimension is $n = 20$ (a *sequence* of inputs).

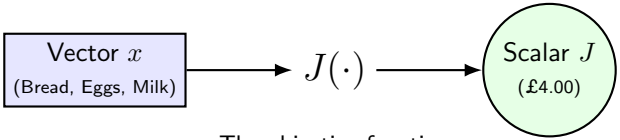
Why $J : \mathbb{R}^n \to \mathbb{R}$? The shopping basket analogy (1/2)

Vector of choices:

$$x = \begin{bmatrix} 2 \\ 6 \\ 1 \end{bmatrix} \quad \leftarrow \text{(Bread, Eggs, Milk)}$$

The till applies a rule (prices) to produce a single number:

$$J(x) = (2 \times £1.00) + (6 \times £0.20) + (1 \times £0.80)$$



The objective function

Why $J : \mathbb{R}^n \to \mathbb{R}$? The shopping basket analogy (2/2)

- You can't easily compare two baskets by looking at item-by-item counts.
- After mapping to a scalar (total price), comparison is immediate.

Key takeaway

Optimisation algorithms cannot minimise a list/vector. They minimise a **single number**. The objective function is the translator from complex engineering decisions to a scalar score.

- In MPC, Plan A may track better but use more fuel than Plan B.
- $J(x)$ compresses all trade-offs so we can rank plans.

Constraints and the feasible set

Constraints define a region of allowable decisions.

- Inequalities: $g_i(x) \leq b_i$ (bounds, safety).
- Equalities: $h_j(x) = 0$ (physics, model dynamics).
- Domain restrictions: $x \in \mathbb{R}^{n-p} \times \mathbb{Z}^p$ (discrete logic) — usually hard.

$$S = \{x \in \mathbb{R}^n \mid g_i(x) \leq b_i \ \forall i, \quad h_j(x) = 0 \ \forall j\}$$

Infeasibility

If constraints conflict and the intersection is empty, the problem has **no solution**.

Infeasibility and empty feasible sets

What if constraints contradict each other?

If the intersection of all constraints is the **empty set**, the problem is **infeasible** — no solution exists.

Example: $x \leq 2$ and $x \geq 5$ simultaneously — no x satisfies both.

In MPC: Infeasibility can arise when:

- The prediction horizon is too short to satisfy terminal constraints.
- State constraints are too tight relative to the disturbance level.
- The system starts too far from the target.

Practical remedy

Use **soft constraints** (add a penalty for violation instead of a hard limit) to ensure the MPC always has a feasible solution.

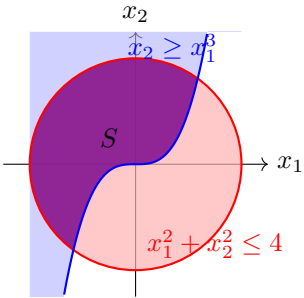
Visualising a feasible set (intersection of constraints)

Example

In $\mathbb{R}^2$, constrain points to be:

1. Inside a circle: $x_1^2 + x_2^2 \leq 4$
2. Above a cubic: $x_2 \geq x_1^3$

Feasible set S is their intersection.



Min vs. argmin (what does the solver return?)

- $\min f(x)$ returns the **value** of the function at the minimum.
- $\arg\min f(x)$ returns the **argument** $x^\star$ that achieves it.

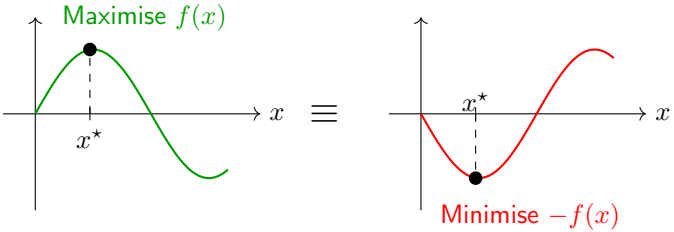
In MPC

We typically care about $\arg\min$ because we need the *input* to apply, not just the cost value.

Maximisation vs. minimisation

Most numerical solvers are written for **minimisation**.

$$\arg\max_{x \in S} f(x) \equiv \arg\min_{x \in S} \{-f(x)\}$$



Why geometry matters

Some optimisation problems are solvable in milliseconds; others can take hours or fail.

- The key distinction is often not just *size*, but **geometry**.
- **Convexity** is the property that makes problems reliable and fast.
- In convex problems: local optimum $\Rightarrow$ global optimum.

Convex sets

Convex Set

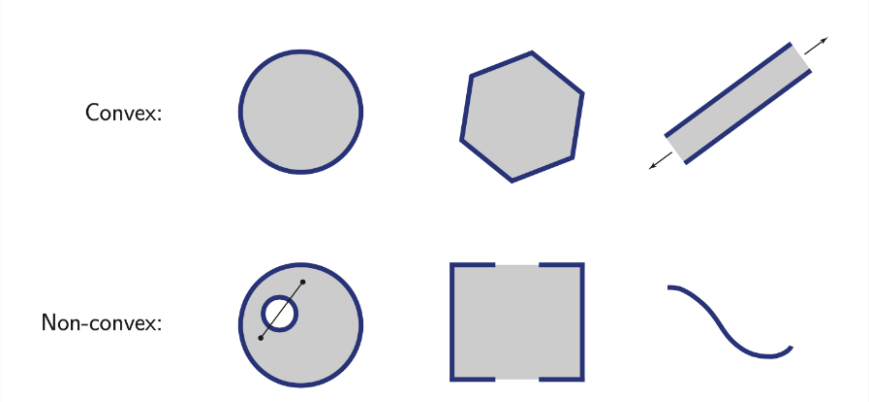
A set $\mathcal{X}$ is **convex** iff for any $x, y \in \mathcal{X}$, every convex combination lies in $\mathcal{X}$:

$$\lambda x + (1 - \lambda) y \in \mathcal{X}, \quad \forall \lambda \in [0, 1].$$

Interpretation

All line segments starting and ending in $\mathcal{X}$ stay within $\mathcal{X}$.

Convex vs non-convex sets (visual)



Rule of thumb

If the line segment between two points in the set ever leaves the set, it is **non-convex**.

Hyperplanes and half-spaces

Hyperplane

A hyperplane is $\{x \in \mathbb{R}^n \mid a^\top x = b\}$, with normal vector $a \neq 0$.

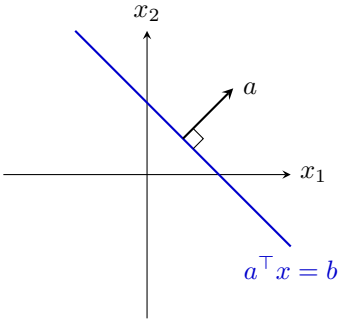
Half-space

A (closed) half-space is $\{x \in \mathbb{R}^n \mid a^\top x \leq b\}$.

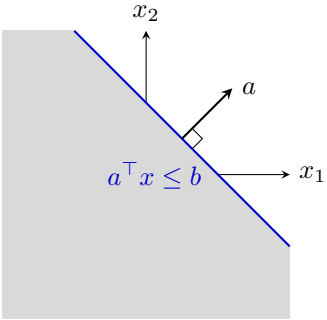
Key property

Hyperplanes and half-spaces are always **convex**. Intersections of half-spaces form polyhedra.

Hyperplane vs half-space (geometry)



Hyperplane



Half-space

Example: linear constraint as a hyperplane

Consider $2x_1 + x_2 = 2$. This matches $a^\top x = b$ with

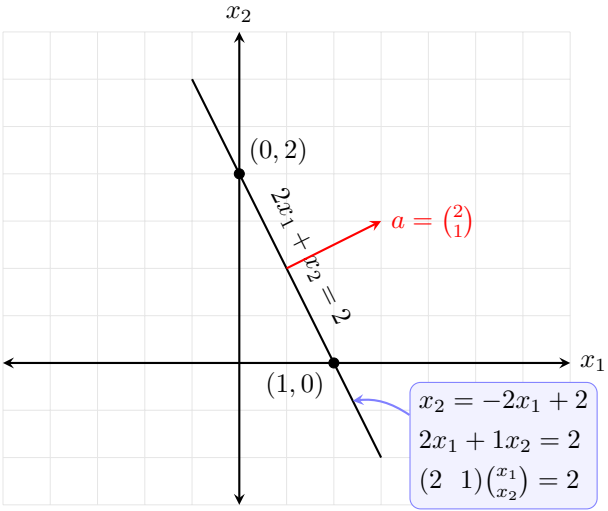
$$a = \begin{pmatrix} 2 \\ 1 \end{pmatrix}, \quad x = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}, \quad b = 2.$$

Gradient connection: define $f(x) = 2x_1 + x_2 - 2$. Then

$$\nabla f = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \end{pmatrix} = \begin{pmatrix} 2 \\ 1 \end{pmatrix} = a,$$

so the gradient is normal to the level set (the line).

Geometry of $2x_1 + x_2 = 2$ and the normal vector



Polyhedra and polytopes

Polyhedron

A polyhedron is an intersection of finitely many closed half-spaces:

$$P = \{x \in \mathbb{R}^n \mid a_i^\top x \leq b_i,\ i = 1, \ldots, m\} = \{x \in \mathbb{R}^n \mid Ax \leq b\}.$$

Polytope

A polytope is a **bounded** polyhedron.

Important observation

Intersections of convex sets are convex. Each half-space is convex $\Rightarrow$ every polyhedron (and polytope) is convex. Unions of convex sets are **not** convex in general.

Numerical example: a polytope in $\mathbb{R}^2$

Constraints:

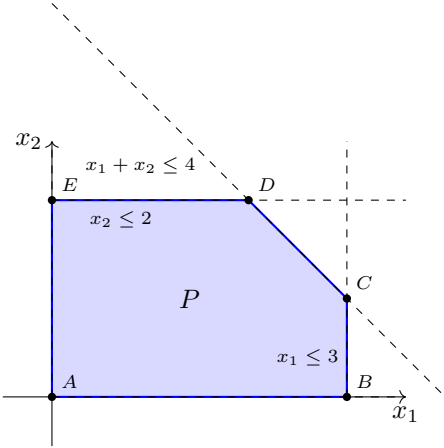
$$x_1 \geq 0, \qquad x_2 \geq 0,$$
$$x_1 \leq 3, \qquad x_2 \leq 2,$$
$$x_1 + x_2 \leq 4.$$

Standard form $Ax \leq b$:

$$A = \begin{bmatrix} -1 & 0 \\ 0 & -1 \\ 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \quad b = \begin{bmatrix} 0 \\ 0 \\ 3 \\ 2 \\ 4 \end{bmatrix}.$$

Vertices: $(0,0), (3,0), (3,1), (2,2), (0,2)$.

Polytope visualisation



Convex functions

Let S be convex.

Convex Function

$f : S \to \mathbb{R}$ is convex if for all $z_1, z_2 \in S$ and $\lambda \in [0, 1]$,

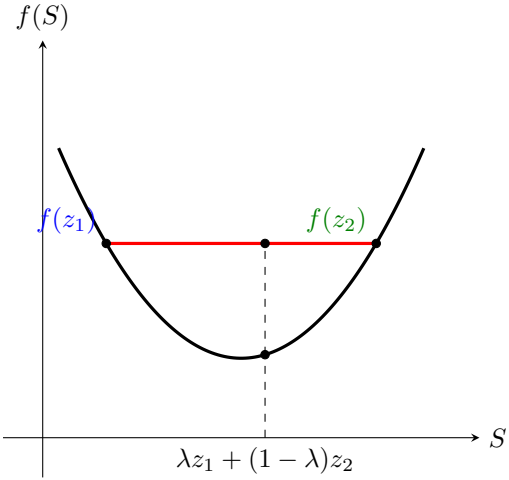
$$f(\lambda z_1 + (1 - \lambda) z_2) \leq \lambda f(z_1) + (1 - \lambda) f(z_2).$$

Geometrically: the chord between two points lies above the graph.

Concave Function

f is concave if $-f$ is convex (inequality reverses).

Geometric picture of convexity



Why convexity matters for MPC

The Fundamental Theorem of Convex Optimisation

If the objective function $J(x)$ is **convex** and the feasible set S is **convex**, then every local minimum is a **global minimum**.

Consequence for MPC:

- Linear MPC with quadratic cost and linear constraints $\Rightarrow$ **convex QP**.
- The solver is *guaranteed* to find the best possible control action.
- No risk of getting "stuck" in a bad local minimum.
- Solution can be found *efficiently* (polynomial time).

When convexity is lost

Nonlinear MPC with a nonconvex model may have multiple local minima. The solver may return a suboptimal solution. This is one of the main challenges of nonlinear MPC.

Two axes of classification

We commonly classify optimisation problems by:

1. **Constraints:** unconstrained vs constrained (equality/inequality/both).
2. **Structure:** linear / quadratic / nonlinear; convex vs nonconvex; continuous vs integer.

Gradient, Hessian, and optimality

Gradient: $\nabla J = \begin{bmatrix} \frac{\partial J}{\partial x_1} & \cdots & \frac{\partial J}{\partial x_n} \end{bmatrix}^\top$ — direction of steepest ascent.

Hessian: $\nabla^2 J$ — matrix of second derivatives. Encodes curvature.

Necessary condition for a minimum: $\nabla J(x^\star) = 0$ (stationary point).

Sufficient condition: $\nabla J(x^\star) = 0$ *and* $\nabla^2 J(x^\star) \succ 0$ (positive definite Hessian).

Connection to convexity

If $\nabla^2 J(x) \succeq 0$ *everywhere*, then J is convex and any stationary point is a global minimum. For a QP with cost $\frac{1}{2}x^\top P x + q^\top x$, the Hessian is P, so convexity $\Leftrightarrow P \succeq 0$.

Unconstrained optimisation

Unconstrained means x can be anywhere in $\mathbb{R}^n$:

$$\min_{x \in \mathbb{R}^n} J(x)$$

Visualisation tool: level curves (contours) $J(x) = c$.

Topographic map analogy

Contours represent constant "height" (cost). Minimisation finds the smallest attainable contour.

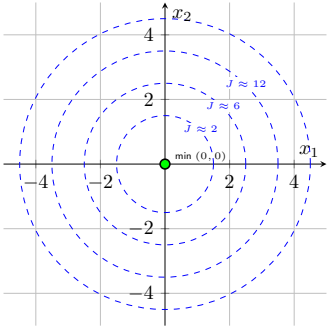
Example: level curves of a bowl

Unconstrained example

$$J(x) = x_1^2 + x_2^2$$

Contours are circles $x_1^2 + x_2^2 = c$.

Level curves of $J(x) = x_1^2 + x_2^2$



Constrained optimisation (types)

We often must enforce limits: $x \in S$.

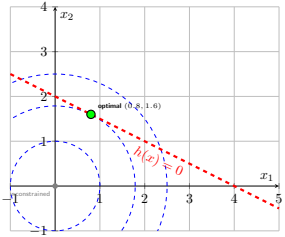
- **Equality constraints:** $h(x) = 0$ restrict to a manifold (curve/surface).
- **Inequality constraints:** $g(x) \leq 0$ restrict to a region.
- **General constraints:** both equality and inequality together.

Type A: equality constraints (tangent point idea)

Problem

$$\min_{x} \quad J(x) = x_1^2 + x_2^2$$
$$\text{s.t.} \quad h(x) = x_1 + 2x_2 - 4 = 0$$

Minimising $J(x)$ on the line $h(x) = 0$



Geometric rule

The constrained optimum occurs where the constraint is tangent to a level curve.

Type B: inequality constraints + YALMIP workflow

Problem

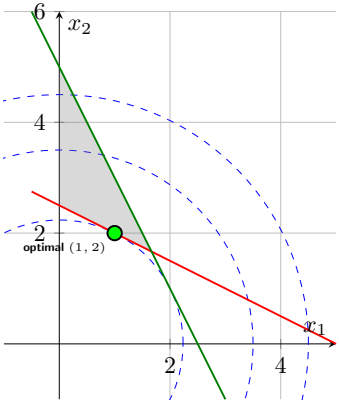
$$\min\ J(x) = x_1^2 + x_2^2 \quad \text{s.t.} \quad \begin{cases} -x_1 - 2x_2 + 5 \leq 0 \\ 2x_1 + x_2 - 5 \leq 0 \\ x_1 \geq 0,\ x_2 \geq 0 \end{cases}$$

YALMIP pattern: Define variables $\rightarrow$ define objective $\rightarrow$ define constraints $\rightarrow$ solve.

Inequality constraints: YALMIP implementation

```
% 1. Define Decision Variables
x = sdpvar(2,1);

% 2. Define Objective
J = x(1)^2 + x(2)^2;

% 3. Define Constraints (vectorised list)
Constraints = [ -x(1) - 2*x(2) + 5 <= 0, ...
                 2*x(1) +   x(2) - 5 <= 0, ...
                 x(1) >= 0, ...
                 x(2) >= 0 ];

% 4. Solve
optimize(Constraints, J);

% 5. Extract Result
x_star = value(x)   % Expected: (1, 2)
```

Inequality constraints: geometric interpretation



Picture

Expand contours from the unconstrained minimum until first contact with the feasible set.

Type C: general constraints (mix equality + inequalities)

Problem

$$
\begin{aligned}
\min \quad & J(x) = x_1^2 + x_2^2 \\
\text{s.t.} \quad & -x_1 - 2x_2 + 5 \le 0, \quad 2x_1 + x_2 - 5 \le 0, \quad x_1 \ge 0, \; x_2 \ge 0 \\
& \underbrace{x_1 + 2x_2 - 7 = 0}_{\text{equality constraint}}
\end{aligned}
$$

MATLAB syntax pitfall

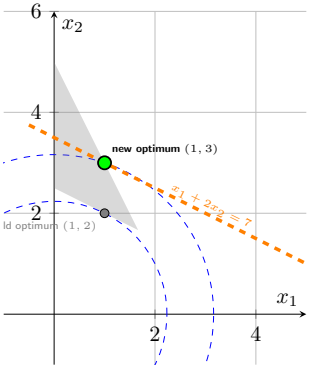
Use == for constraints; = is assignment.

General constraints: YALMIP implementation

```
% 1. Variables and Objective
x = sdpvar(2,1);
J = x(1)^2 + x(2)^2;

% 2. Inequalities (region)
g1 = [-x(1) - 2*x(2) + 5 <= 0];
g2 = [ 2*x(1) +   x(2) - 5 <= 0];
g3 = [x(1) >= 0];
g4 = [x(2) >= 0];

% 3. Equality (manifold)
h  = [x(1) + 2*x(2) - 7 == 0];

% 4. Solve
Constraints = [g1, g2, g3, g4, h];
optimize(Constraints, J);

x_star = value(x)   % Expected: (1, 3)
```

General constraints: geometry + "cost of constraints"

Intersection of equality and inequalities



The cost of constraints

Adding constraints restricts freedom and typically **increases** the minimum achievable cost.

Problem classes (structure)

- **LP:** linear objective + linear constraints (fast, robust).
- **QP:** quadratic objective + linear constraints (standard linear MPC).
- **Nonconvex QP:** indefinite quadratic (harder; local minima).
- **MILP/MIP:** integer variables + linear structure (combinatorial; expensive).

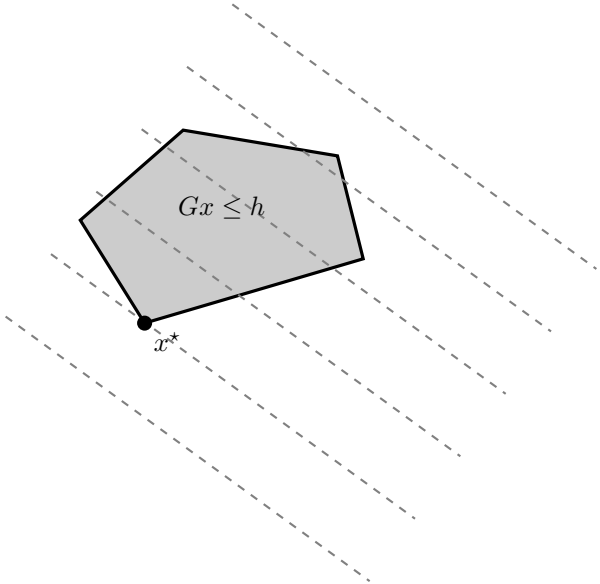
Linear programmes (LPs)

Standard form:

$$\begin{aligned} \min_{x} \quad & c^\top x \\ \text{s.t.} \quad & Gx \leq h \\ & Ax = b \end{aligned}$$

- Feasible set is a polyhedron.
- If an optimum exists, it occurs at an extreme point (vertex).

LP geometry: shifting cost hyperplanes



LP example: minimum-effort steady state

Goal

For $x_{k+1} = Ax_k + Bu_k$, find an equilibrium $(x_{\text{ss}}, u_{\text{ss}})$ satisfying constraints, with minimum control effort (fuel).

Effort as 1-norm:

$$\|u\|_1 = \sum_{i=1}^{m} |u_i|$$

Epigraph trick: introduce $t \in \mathbb{R}^m$ with $-t \leq u \leq t$ and minimise $\mathbf{1}^\top t$.

LP example: full formulation (with norm linearisation)

$$
\begin{aligned}
\min_{u,\, x_{\mathrm{ss}},\, t} \quad & \mathbf{1}^\top t \\
\text{s.t.} \quad & (I - A)x_{\mathrm{ss}} - Bu = 0 \qquad \text{(equilibrium)} \\
& Gx_{\mathrm{ss}} \le h \qquad \text{(safety)} \\
& u_{\min} \le u \le u_{\max} \qquad \text{(actuator)} \\
& -t \le u \le t \qquad \text{(linearise } |u|\text{)}
\end{aligned}
$$

Why this matters in MPC

Many "nonlinear-looking" penalties can be turned into LP/QP-friendly forms by adding auxiliary variables.

Stigler's Diet Problem (2D simplified)

Decision variables

$x_1 =$ Corn [kg], $x_2 =$ Wheat Bread [kg].

Objective: minimise cost $J = c_1 x_1 + c_2 x_2$.

Constraints:

- Energy (kcal): $K_{\min} \leq k_1 x_1 + k_2 x_2 \leq K_{\max}$
- Vitamin A (μg): $V_{\min} \leq v_1 x_1 + v_2 x_2 \leq V_{\max}$
- Nonnegativity: $x_1, x_2 \geq 0$

Stigler data (this simplified instance)

Data:

- Costs: $c_1 = 2.0$, $c_2 = 0.5$ [\$/kg]
- Energy: $k_1 = 750$, $k_2 = 650$ (target: 2000–3000 kcal)
- Vitamin A: $v_1 = 250$, $v_2 = 0$ (target: 500–800 μg)

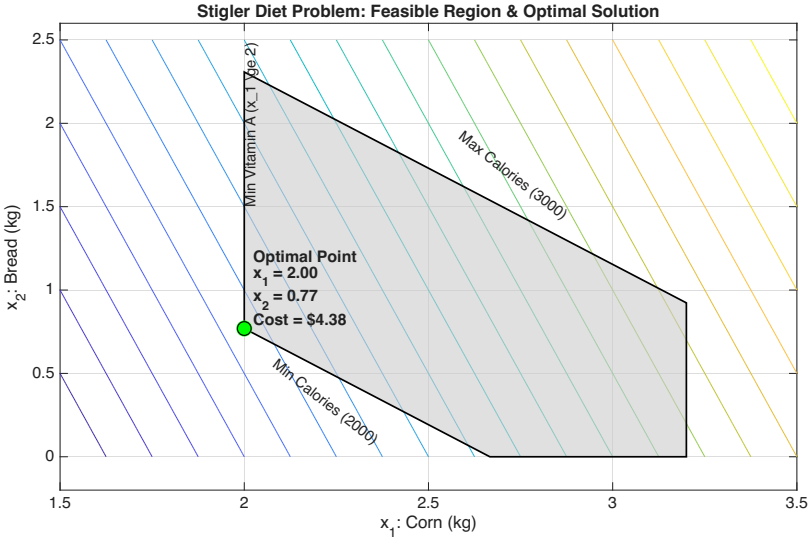
$$
\begin{aligned}
\min_{x_1, x_2} \quad & 2x_1 + 0.5x_2 \\
\text{s.t.} \quad & 2000 \leq 750x_1 + 650x_2 \leq 3000 \\
& 500 \leq 250x_1 \leq 800 \\
& x_1 \geq 0, \; x_2 \geq 0
\end{aligned}
$$

Stigler in MATLAB/YALMIP

Listing 1: Solving Stigler's Problem with YALMIP

```
clear; clc;

x1 = sdpvar(1); % Corn (kg)
x2 = sdpvar(1); % Bread (kg)

Objective = 2*x1 + 0.5*x2;

Constraints = [];
Constraints = [Constraints, 2000 <= 750*x1 + 650*x2 <= 3000];
Constraints = [Constraints, 500  <= 250*x1           <= 800 ];
Constraints = [Constraints, x1 >= 0, x2 >= 0];

options = sdpsettings('verbose', 0, 'solver', 'linprog');
sol = optimize(Constraints, Objective, options);

if sol.problem == 0
  fprintf('x1 (Corn) : %.4f kg\n', value(x1));
  fprintf('x2 (Bread): %.4f kg\n', value(x2));
  fprintf('Min Cost  : $%.4f\n', value(Objective));
else
  disp('Problem could not be solved.');
end
```

Stigler: feasible set and optimum (your provided figure)



Convex quadratic programmes (QPs)

Standard QP:

$$\begin{aligned} \min_x \quad & \frac{1}{2} x^\top P x + q^\top x \\ \text{s.t.} \quad & Gx \leq h \\ & Ax = b \end{aligned}$$

Convex iff $P \succeq 0$ (positive semidefinite).

How to test $P \succeq 0$ (positive semidefinite)

Equivalent tests (for symmetric P)

1. All eigenvalues are nonnegative: $\lambda_i(P) \geq 0$.
2. Quadratic form is nonnegative: $z^\top P z \geq 0, \ \forall z$.
3. All leading principal minors have nonnegative determinant.

The Hessian

Hessian

For $f : \mathbb{R}^n \to \mathbb{R}$, the Hessian $H(x) = \nabla^2 f(x)$ collects second partial derivatives:

$$H_{ij} = \frac{\partial^2 f}{\partial x_i \, \partial x_j}.$$

For smooth f, H is symmetric.

Example: computing a Hessian

For

$$f(x) = 3x_1^2 + 2x_1x_2 + 5x_2^2,$$

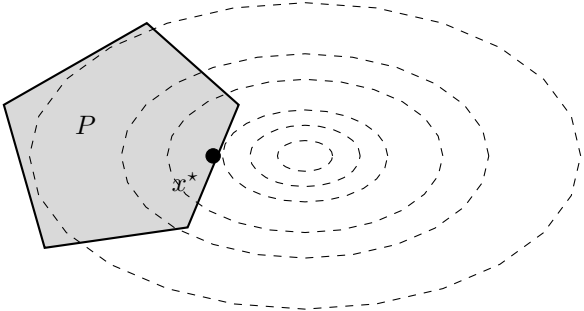
Gradient:

$$\frac{\partial f}{\partial x_1} = 6x_1 + 2x_2, \qquad \frac{\partial f}{\partial x_2} = 2x_1 + 10x_2.$$

Hessian:

$$H = \begin{bmatrix} 6 & 2 \\ 2 & 10 \end{bmatrix}.$$

QP geometry: ellipses + polytope



Interpretation

For strictly convex $P \succ 0$, the global minimiser is unique (if feasible set nonempty).

QP example: distance to a restricted zone (setup)

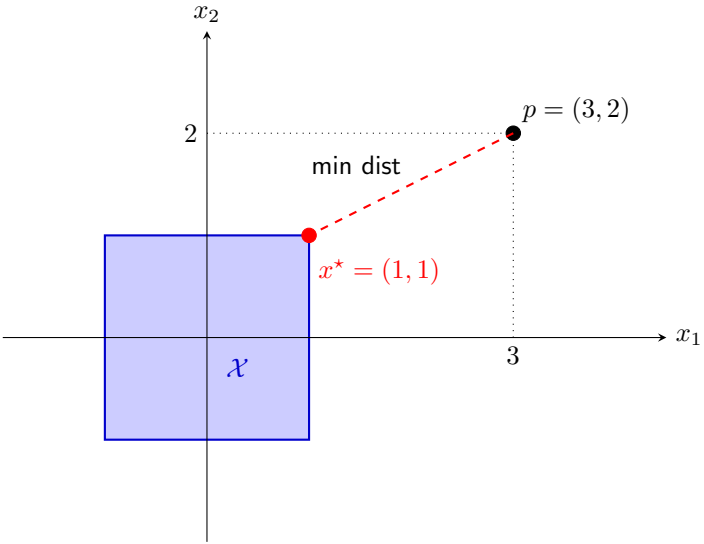
Scenario

Robot at $p = (3, 2)$. Restricted zone $\mathcal{X}$ is a unit box centered at origin: $-1 \leq x_1 \leq 1$, $-1 \leq x_2 \leq 1$.

Goal: find closest point in the box:

$$\min_{x \in \mathcal{X}} (x_1 - 3)^2 + (x_2 - 2)^2.$$

QP example: geometry (closest point)



QP example: derive P, q, G, h (standard form)

Expand objective:

$$\begin{aligned} J(x) &= (x_1 - 3)^2 + (x_2 - 2)^2 \\ &= x_1^2 + x_2^2 - 6x_1 - 4x_2 + 13. \end{aligned}$$

Drop constant 13 and match $\frac{1}{2}x^\top P x + q^\top x$:

$$P = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}, \qquad q = \begin{bmatrix} -6 \\ -4 \end{bmatrix}.$$

Box constraints $-1 \le x_i \le 1$ become $Gx \le h$:

$$G = \begin{bmatrix} 1 & 0 \\ -1 & 0 \\ 0 & 1 \\ 0 & -1 \end{bmatrix}, \qquad h = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}.$$

QP example: MATLAB/YALMIP (quadprog backend)

Listing 2: Solving the robot distance QP using YALMIP + quadprog.

```
clear; clc;
ops = sdpsettings('solver', 'quadprog', 'verbose', 0);

p = [3; 2];            % Robot position
x = sdpvar(2, 1);      % Decision variable

Objective  = (x(1) - p(1))^2 + (x(2) - p(2))^2;
Constraints = [-1 <= x <= 1];

optimize(Constraints , Objective, ops);

x_opt = value(x);
fprintf('Optimal Sol : x1=%.2f, x2=%.2f\n', x_opt(1), x_opt(2));
```

QP example: direct quadprog matrices (opening the black box)

MATLAB `quadprog` expects:

$$\min_x \ \frac{1}{2}x^\top H x + f^\top x \quad \text{s.t.} \quad Ax \le b.$$

Listing 3: Direct solution using MATLAB's quadprog.

```
clear; clc;

% Objective: 1/2*x'*P*x + q'*x
H = [2, 0;
     0, 2];            % P
f = [-6; -4];          % q

% Constraints: G*x <= h
A = [ 1,  0;
     -1,  0;
      0,  1;
      0, -1];          % G
b = [1; 1; 1; 1];      % h

options = optimoptions('quadprog', 'Display', 'off');
x_opt = quadprog(H, f, A, b, [], [], [], [], [], options);

fprintf('Optimal Sol : x1=%.2f, x2=%.2f\n', x_opt(1), x_opt(2));
```

The QP is the workhorse of linear MPC

Why QPs dominate MPC

Linear MPC with a quadratic cost function and linear constraints produces a **convex QP** at every time step:

$$\min_{U} \ \frac{1}{2} U^\top H\, U + f^\top U \quad \text{s.t.} \quad G\, U \leq h$$

where $U = [u_0^\top, u_1^\top, \ldots, u_{N-1}^\top]^\top$ is the stacked input sequence.

Key properties:

- $H \succ 0$ (positive definite) $\Rightarrow$ unique global minimum exists.
- QP solvers (`quadprog`, OSQP, qpOASES) can solve problems with hundreds of variables in milliseconds.
- The QP structure is *the same* at every time step — only the parameters change (current state x_k).
- This enables **warm-starting**: using the previous solution as an initial guess.

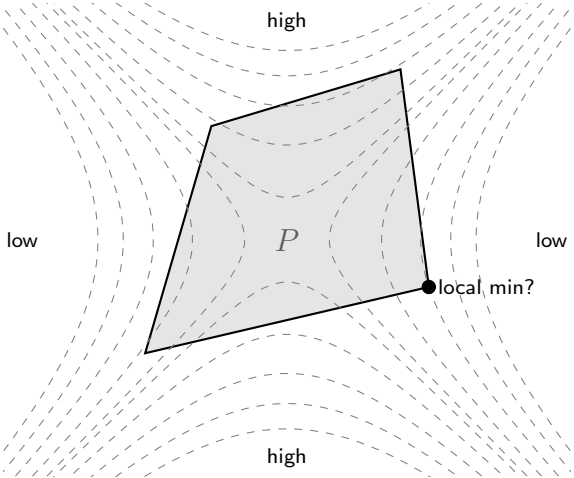
Nonconvex quadratic programmes

If $P \not\succeq 0$ (indefinite), the objective is nonconvex:

$$\min_x \ \frac{1}{2}x^\top P x + q^\top x \quad \text{s.t.} \quad Gx \leq h, \ Ax = b.$$

- May have multiple local minima and saddle points.
- Global optimality is hard to guarantee efficiently.
- Standard linear MPC avoids this by using convex QPs.

Nonconvex QP: saddle-shaped contours (sketch)



Mixed-integer linear programmes (MILPs)

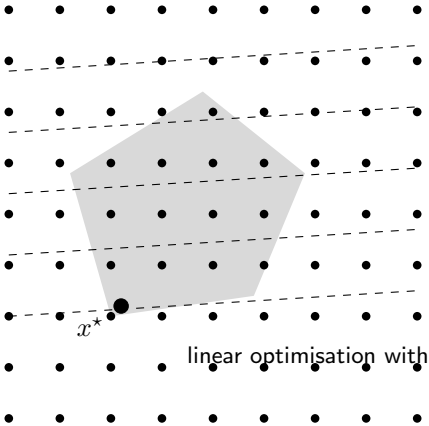
MILP: linear objective + linear constraints, but some variables are integers/binary:

$$\begin{aligned} \min_{x} \quad & c^\top x \\ \text{s.t.} \quad & Gx \le h, \;\; Ax = b \\ & x \in \{0,1\}^p \times \mathbb{R}^{n-p} \;\; \text{or} \;\; x \in \mathbb{Z}^p \times \mathbb{R}^{n-p}. \end{aligned}$$

Why hard?

Feasible set becomes **discrete points** in a polyhedron $\Rightarrow$ combinatorial explosion.

MILP sketch: integer lattice inside a polytope



linear optimisation with integer constraints

MILP example: industrial energy management (model structure)

Idea

Grid + generator + battery. Minimise operating cost while meeting demand.

At each time step $k = 1, \ldots, N$, typical variables include:

- Continuous: $p_g(k), p_{\text{grid}}(k), p_{\text{ch}}(k), p_{\text{dis}}(k), E(k)$
- Binary logic: $u(k)$ on/off, $s(k)$ start-up, $z(k)$ charge-mode

MILP drivers

Binary variables make the problem mixed-integer $\Rightarrow$ need solvers like Gurobi/CPLEX.

Two-layer software stack

1. **Parsers / modelling languages:** translate symbolic constraints/objectives to matrices.
2. **Solvers:** numerical engines (active-set, interior point, ADMM, etc.).

Why you should care

Understanding the mapping from maths $\rightarrow$ matrices $\rightarrow$ solver inputs helps you debug and design MPC effectively.

Modelling languages (parsers)

Name	Platform	License	Primary purpose / notes
YALMIP	MATLAB	Free	Flexible modelling for LP/QP/SDP/MIP; great for prototyping.
CVX	MATLAB	Free	Disciplined convex programming (DCP); strong convexity guarantees.
CVXPY	Python	Free (Apache)	Python equivalent of CVX; clean OO interface for convex problems.
CasADi	Python/C++/MATLAB	Free (LGPL)	Nonlinear MPC; algorithmic differentiation; C-code generation.
Pyomo	Python	Free (BSD)	General operations research modelling; wide solver support.
JuMP	Julia	Free (MIT)	Very fast modelling; popular in optimisation/control community.

Numerical solvers (what they are good for)

Solver	Type	Method	License	Best for
quadprog	QP	Interior-point	Proprietary	Standard MATLAB QP (course default; Opt Toolbox).
OSQP	QP	ADMM	Free (Apache)	Embedded MPC; robust, fast to medium accuracy.
qpOASES	QP	Active-set	Free (LGPL)	MPC warm-starting; small/medium dense QPs.
Gurobi	LP/QP/MIP	Barrier/Simplex	Commercial	Very fast large-scale convex + MIP (acad. licenses).
MOSEK	SOCP/SDP	Interior-point	Commercial	Conic/robust MPC; LMIs/SDP.
SeDuMi	SOCP/SDP	Interior-point	Free (GPL)	Classic MATLAB LMI/SDP solver.
IPOPT	NLP	Interior-point	Free (EPL)	Large-scale nonlinear optimisation (NMPC).
SNOPT	NLP	SQP	Commercial	Difficult nonconvex NLPs (e.g. trajectories).
ECOS	SOCP/LP	Interior-point	Free (GPL)	Lightweight conic solver, embedded friendly.

YALMIP: the five-step workflow

1. **Define variables:** `x = sdpvar(n, 1)`
2. **Define objective:** `J = x'*Q*x + c'*x`
3. **Define constraints:** `C = [A*x <= b, x >= 0]`
4. **Solve:** `optimize(C, J, sdpsettings('solver','quadprog'))`
5. **Extract solution:** `x_opt = value(x)`

Why use YALMIP instead of calling solvers directly?

- You write maths, YALMIP writes matrices — fewer bugs.
- Same code works with different solvers (just change the solver name).
- Automatic detection of problem type (LP, QP, SDP, ...).
- The `optimizer` object pre-compiles the problem for fast repeated solves (essential for MPC).

Economic MPC for smart grids (battery arbitrage)

Scenario

Home with load P_{load}, solar P_{solar}, battery energy E_{bat}, grid exchange P_{grid}. Prices vary throughout the day.

Goal: choose charge/discharge schedule to minimise daily electricity cost over $N = 24$ hours.

Model (variables, objective, constraints)

Let $\Delta t = 1$ hour and $k = 0, \ldots, N-1$.

Variables:

- $P_{\text{grid}}(k)$: grid power (+buy, -sell)
- $P_{\text{bat}}(k)$: battery power (+charge, -discharge)
- $E_{\text{bat}}(k)$: stored energy

Objective:

$$\min \sum_{k=0}^{N-1} \text{Price}(k)\, P_{\text{grid}}(k)\, \Delta t$$

Constraints:

$$\begin{aligned} P_{\text{grid}}(k) + P_{\text{solar}}(k) &= P_{\text{load}}(k) + P_{\text{bat}}(k) \\ E_{\text{bat}}(k+1) &= E_{\text{bat}}(k) + P_{\text{bat}}(k)\Delta t \\ E_{\min} \le E_{\text{bat}}(k) \le E_{\max}, &\qquad P_{\min} \le P_{\text{bat}}(k) \le P_{\max} \end{aligned}$$

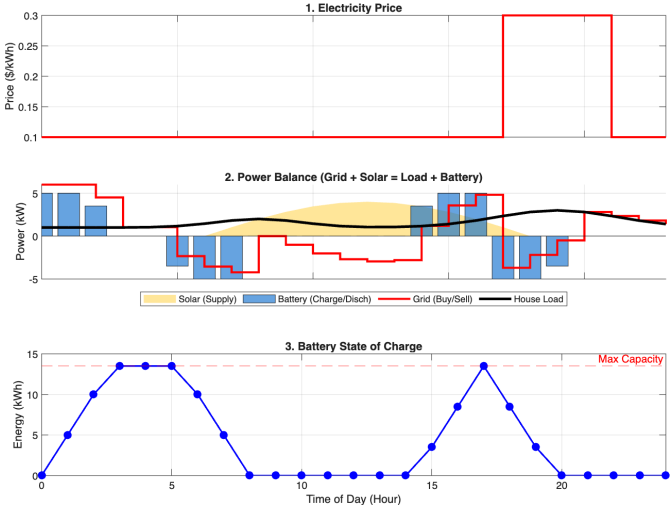
MATLAB/YALMIP implementation (battery arbitrage LP) 1/2

```
% --- 1. System Parameters ---
N = 24;                    % Horizon (24 hours)
E_cap = 13.5;              % Battery Capacity (kWh)
P_max = 5;                 % Max Charge/Discharge rate (kW)
dt = 1;                    % Time step (1 hour)

% --- 2. Forecast Data (Profiles) ---
Price = 0.10 * ones(N,1);
Price(18:21) = 0.30;       % Peak 17:00--21:00 (1-indexed hours)

t = (0:N-1)';
Solar = max(0, 4 * sin(pi * (t - 6) / 12));  % simple solar profile
Load  = 1 + 2 * exp(-(t-19).^2/10) + 1 * exp(-(t-8).^2/5);

% --- 3. Optimisation Variables ---
P_grid = sdpvar(N, 1);
P_bat  = sdpvar(N, 1);
E_bat  = sdpvar(N+1, 1);
```

MATLAB/YALMIP implementation (battery arbitrage LP) 2/2

```
      :
      :
      :

% --- 4. Constraints ---
C = [];
C = [C, E_bat(1) == 0];  % initial SoC (in MPC use measurement)

for k = 1:N
C = [C, E_bat(k+1) == E_bat(k) + P_bat(k)*dt];
C = [C, P_grid(k) + Solar(k) == Load(k) + P_bat(k)];
C = [C, 0 <= E_bat(k+1) <= E_cap];
C = [C, -P_max <= P_bat(k) <= P_max];
end

% --- 5. Objective ---
Objective = Price' * P_grid;

% --- 6. Solve ---
ops = sdpsettings('solver','quadprog','verbose',0);
sol = optimize(C, Objective, ops);
```

Result visualisation (your provided figure)



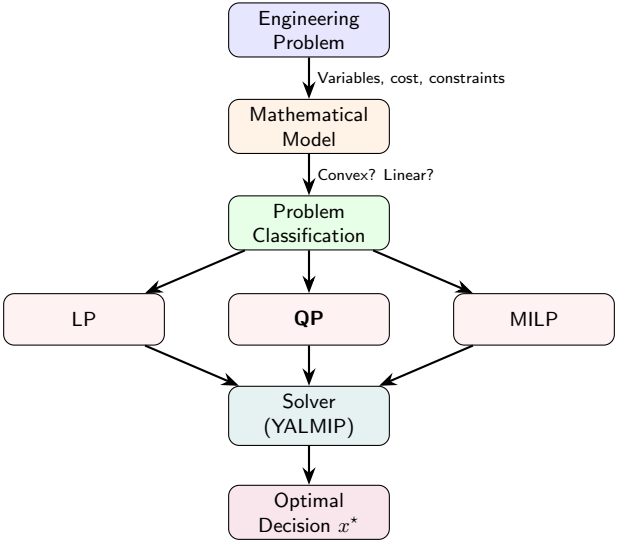
Interpreting the result

1. **Pre-charging:** at low prices (night), the optimiser buys electricity to charge the battery.
2. **Peak shaving / arbitrage:** during high price window, it discharges (and may sell) to avoid expensive grid usage.
3. **Constraint realism:** E_{bat} and P_{bat} limits shape what is achievable.

MPC connection

In closed-loop MPC, the forecast profiles are updated each step and the initial $E_{\mathrm{bat}}(1)$ comes from sensors.

Chapter summary: the optimisation landscape



Key takeaways (1/2)

- **Optimisation is translation:** from qualitative goals to quantitative maths.
- **Scalar score:** algorithms minimise a single number; $J(x)$ ranks plans.
- **Constraints define reality:** feasibility is the intersection of all constraints.
- **Infeasible** problems have no solution; modelling matters.

Key takeaways (2/2)

- **Convexity is king:** convex objective + convex feasible set $\Rightarrow$ local min is global min.
- **Standard forms matter:**
  - LP: very fast; economic dispatch / simple regulation.
  - QP: standard linear MPC (energy/error costs) with linear constraints.
  - MILP: logic/on-off decisions; powerful but computationally expensive.
- **Max $\leftrightarrow$ min:** $\arg\max f(x) \equiv \arg\min(-f(x))$.

Next steps (bridge to MPC formulations)

- In MPC, decision vector becomes a stacked input sequence $U = [u_0^\top, u_1^\top, \ldots, u_{N-1}^\top]^\top$.
- Dynamics appear as equality constraints across the horizon.
- Standard linear MPC leads to a **convex QP** solved every sampling time.
- Warm-starting and solver selection are practical performance levers.

Exercises for self-study (1/2)

Exercise 1: Convex sets and functions.

- Determine whether the following sets are convex:
  1. $\{x \in \mathbb{R}^2 \mid x_1^2 + x_2^2 \leq 4\}$
  2. $\{x \in \mathbb{R}^2 \mid x_1^2 + x_2^2 \geq 1\}$
  3. $\{x \in \mathbb{R}^2 \mid 2x_1 + 3x_2 \leq 6,\ x_1 \geq 0,\ x_2 \geq 0\}$
- Check whether $f(x) = 5x_1^2 - 4x_1x_2 + 3x_2^2$ is convex by computing the Hessian eigenvalues.

Exercise 2: Gradient and Hessian.

- For $J(x) = 4x_1^2 + 6x_1x_2 + 8x_2^2 - 10x_1 + 2x_2$:
  1. Compute ∇J, set it to zero, and find the stationary point.
  2. Compute $\nabla^2 J$. Is J convex?

Exercises for self-study (2/2)

Exercise 3: LP formulation.

- A factory makes products A and B. Product A needs 2h on Machine 1 and 3h on Machine 2; product B needs 4h on Machine 1 and 1h on Machine 2. Available: 10h (M1), 12h (M2). Profit: £3/unit (A), £5/unit (B). Formulate as LP.

Exercise 4: QP standard form.

- For $\min_u 3u_1^2 + 2u_1u_2 + 4u_2^2 - 8u_1 - 6u_2$ subject to $u_1 + u_2 \leq 5$, $0 \leq u_1 \leq 3$, $0 \leq u_2 \leq 4$:
  1. Write in standard QP form $\frac{1}{2}u^\top Pu + q^\top u$.
  2. Show $P \succ 0$ (convex). Express constraints as $Gu \leq h$.
  3. Solve using YALMIP or `quadprog`.

Exercise 5: Satellite fuel minimisation.

- Dynamics: $x_{k+1} = Ax_k + Bu_k$. Goal: reach steady state with $\min \|u_{ss}\|_1$, $|u_{ss}| \leq 10$.
- Show the equilibrium condition, apply the epigraph trick, formulate as LP.